项目总结报告

项目总结报告

日期：7 月 30 日

组号	第五组	项目名称	Arxiv-Paper Mate
编程语言	js,python	开发平台和框架	Fastapi, React

项目工作小结

1. 项目实现了哪些功能？简要一一列出。

- 用户注册、登录、JWT 鉴权、个人资料和偏好设置。
- 论文自动抓取与管理：

  定时从 arXiv 拉取指定学科或关键词范围内的论文，支持按主题、时间等维度分类入库，并进行去重。

  支持展示论文的标题、作者、摘要、类别、发布时间、链接等基本信息。论文列表、详情、筛选、搜索、删除和批量管理。
- 论文结构化解析与知识抽取：

  解析论文 PDF 内容，自动生成 `summary.md` 、 `concept.md` 、 `methods.md` 等结构化内容。

  从论文中抽取核心概念、研究问题、方法、实验结论等信息，并建立概念之间的关联关系。
- 论文 Wiki 与检索：

  构建论文知识 Wiki，支持按照标题、作者、关键词方式进行检索。

  支持查看论文之间的引用关系、主题关联和概念关联。
- 智能问答：

  支持用户使用自然语言围绕论文内容、概念解释进行提问。

  系统基于论文知识库返回答案，并给出对应的已解析论文出处或相关内容定位。
- 学习管理：

  支持用户对论文进行收藏、笔记和对比阅读。

  支持记录用户的阅读历史和关注主题。
- 研究脉络分析：自动生成某一研究主题下的发展脉络图、代表性论文列表和研究趋势分析。
- 个性化推荐与订阅：支持按研究主题订阅，每日推送新增论文摘要，并根据用户阅读历史推荐相关论文。

2. 项目实现了哪些非功能？简要一一列出。

- 客户端为Web浏览器。
- 兼容性：适应用于客户端不同的分辨率/尺寸。
- 准确率：解析、检索和问答的准确率大于90%。
- 性能：系统应能支持不少于100条数据记录（如论文元数据等），在100并发的场景下，检索、问答和学习管理的响应时间<3s（不算LLM响应速度）

3. 项目有哪些特点和亮点？例如业务特色、架构风格、设计模式、模型、算法、项目协同等

- 将论文抓取、AI 解析、知识 Wiki、问答和学习管理整合为完整的论文阅读流程。
- 采用 React、FastAPI、PostgreSQL、pgvector、Redis 和 Celery 的前后端分离架构。
- 使用关键词检索与向量语义检索相结合的混合召回和融合排序方案。
- 通过统一 LLM Gateway 封装 DeepSeek 等模型调用，降低模型替换成本。
- 推荐算法综合语义相似度、关键词、主题分类、订阅和内容时效性。
- 项目按五人职责划分模块，并采用功能分支、Pull Request 和 CI 协同开发。

4. 项目的质量控制如何？简要一一列出。例如软件评审、单元测试、API测试、系统的功能测试、性能测试、兼容性测试、界面测试等等

- 通过需求评审、用例图、部署视图和接口讨论检查设计完整性。
- 使用 Ruff 等工具进行 Python 静态检查和代码风格检查。
- 使用 pytest 对认证、论文、学习管理、推荐和订阅等模块进行单元测试。
- 对 REST API 的正常请求、异常请求、权限和响应结构进行接口测试。
- 按测试用例检查抓取、解析、检索、问答、收藏和笔记等功能。
- 设计并执行并发量、响应时间和任务处理能力等性能测试。
- 在不同浏览器和分辨率下进行兼容性、响应式布局和界面检查。
- 使用 GitHub Pull Request 和 CI 检查降低合并冲突与回归风险。

5.大模型工具辅助完成了哪些方面的软件开发实践活动?

- 辅助分析需求、划分系统模块并设计前后端接口。
- 辅助生成项目脚手架、数据模型、接口、业务服务和测试代码。
- 辅助排查接口错误、数据库连接问题、前后端字段不一致及 Git 冲突。
- 辅助论文摘要、概念、方法、实验结论和概念关系抽取。
- 辅助设计混合检索、RAG、推荐排序和提示词方案。

6．大作业里，哪些代码开发、任务环节无法由大模型独立实现，需要人机协作或纯人工完成?

- 部署到云服务器的过程由人工完成
- 需求优先级、业务规则和最终验收标准必须由团队讨论确定。
- LLM 抽取结果是否符合论文原文，需要人工阅读、标注和抽样核对。
- 页面易用性、视觉效果和真实用户体验需要人工操作与评价。
- 云服务器、数据库权限、密钥和网络安全配置需要人工授权与管理。
- 多人代码冲突的业务含义、数据迁移风险和最终合并版本需要开发者判断。
- 性能、兼容性和系统测试需要在真实环境中执行，不能仅依赖大模型推断。

项目组成员以及大模型工具对项目的贡献度（%）

注： 项目管理包括项目计划、报告、沟通与协调等。

姓名	需求	设计	编码	测试	项目管理	小计
王奕博	4	4	5	3	4	20
刘翔	3	5	5	3	4	20
王文杰	4	4	5	3	4	20
陈熹朗	5	3	4	4	4	20
李悦朗	3	5	5	4	3	20

经验、教训和建议

例如大模型辅助开发的经验、教训和建议

一、开发经验

1. **架构清晰，分层严格**

   体现: 前端 pages→services→data/http、后端 API→Services→Models，单向依赖，职责边界明确

2. **Mock 与真实后端无缝切换**

   体现: VITE_API_MODE=mock/http，前端可独立开发调试，后端接口变化只改 Service 层

3. **AI 全链路深度集成设计**

体现: LLM 嵌入摘要抽取、知识提取、RAG 问答、论文对比、概念关系推断等每个核心环节

4. **状态机 + 可恢复流水线**

   体现: 论文分析流水线用完整状态机（含 11种状态），支持心跳检测、僵尸恢复、指数退避重试、版本追踪

5. **降级策略**

   体现: 向量检索失败 → 回退关键词检索 → LLM 不可用 → 模板回答；无证据时 LLM用自身知识回答

6. **统一错误处理**

   体现: 全局统一的 AppError 异常体系，JSON 格式固定为 {code, message,details}，前端可统一处理

7. **统一的 API 响应格式**

   体现: success() 包装所有响应，前端 apiClient.js 统一处理编解码、Query 参数、HTTP 错误

8. **全面测试覆盖**

   体现: 33 个测试文件，覆盖 API 层、Service 层、核心工具、分析流水线、arXiv抓取、安全、RAG 等

9. **CI/CD 自动化**

   体现: GitHub Actions 自动运行 Ruff + pytest，保证代码质量

10. **性能测试脚本**

    体现: 独立的 performance_test.py，可测试 100 并发、p95 < 3s 的性能指标

二、开发教训

1. **前期过度依赖 Mock 数据。** Mock 数据使界面能够快速展示，但也掩盖了鉴权、数据持久化、字段映射和跨页面状态同步问题。
2. **接口契约确定得不够早。** 不同成员独立设计字段后，曾出现命名、数据类型、时间格式和响应层级不一致，增加了联调成本。
3. **数据库模型变更缺少充分协调。** 多名成员同时修改用户、论文和学习管理模型，容易产生迁移冲突以及关联关系不一致。
4. **概念规范化设计不足。** 同一个概念可能以单数、复数、缩写或带括号名称重复入库，影响概念去重、跨论文聚合和关系分析。
5. **知识来源定位设计不完整。** 系统能够保存部分论文和关系证据，但缺少统一的页码、章节、段落或 `chunk_index`，导致前端无法精确定位原文。
6. **部分功能只有展示层完成。** 某些按钮和页面早期只有前端效果，没有对应的后端接口或持久化逻辑，容易让界面看似可用但实际操作无效。
7. **推荐数据与论文库边界不清晰。** 推送论文尚未加入个人论文库时，直接进入本地详情会出现“论文不存在”，后来才明确“阅读原文”和“加入论文库”的不同流程。
8. **时间和时区处理不统一。** 后端 UTC 时间与前端本地时间混用，曾造成阅读动态显示相差八小时的问题。

三、改进建议

1. **建立统一的接口契约。** 使用 OpenAPI 作为前后端共同标准，明确字段名称、类型、是否可空、错误码和示例数据。
2. **增加 Repository 数据访问层。** 将复杂查询和数据持久化从 Service 中分离，便于测试、复用和后续更换存储方案。
3. **完善概念规范化流程。** 对概念进行小写化、单复数处理、缩写展开、别名归并和向量相似度聚类，并允许人工确认合并。
4. **补全知识来源定位。** 为概念、方法、结论和关系证据保存 `paper_id`、页码、章节、段落及 `chunk_index`，并在前端提供跳转入口。
5. **加强跨论文知识关联。** 在规范化概念基础上，根据共同概念、引用关系和语义相似度建立跨论文关系，而不只依赖单篇论文的 LLM 抽取。
6. **统一时间标准。** 数据库统一存储 UTC 时间，API 返回带时区的 ISO 8601 字符串，前端仅在展示时转换为用户本地时间。
7. **加强安全管理。** 密钥、数据库密码和 JWT 密钥只通过环境变量管理，定期更换凭据，并限制数据库网络访问范围。
8. **完善任务可靠性。** 为抓取和解析任务增加超时、重试、幂等控制、失败原因、执行日志和任务状态查询。
9. **明确完成标准。** 每项功能应同时满足前端可操作、后端接口可用、数据可持久化、异常可处理、测试已通过和文档已更新。
10. **建立 LLM 降级机制。** 当模型调用失败或输出不符合结构时，应支持重试、格式修复、任务失败标记以及仅展示基础元数据。
11. **逐步完善部署方式。** 后续可将前端、FastAPI、Redis 和后台任务统一部署到稳定应用服务器，并通过 HTTPS、日志监控和备份提高可用性。

项目组各成员签字：